Tampere University Unit of Computing Sciences
COMP.SE.610 Software Engineering Project 1
COMP.SE.620 Software Engineering Project 2

G05

Snowledge

Test Plan

Saku Hakamäki
Eero Kainulainen
Toivo Knuutinen
Kaapo Kontinen
Eetu Koskela
Huyen Pham
Eemeli Ruohomäki

Version history

Version	Date	Author	Description
1.0	31.10.2025	Saku Hakamäki Eetu Koskela Toivo Knuutinen	First version.
2.0	11.11.2025	Saku Hakamäki Eetu Koskela	Ch. 4.4 → Add last 3 paragraphs to Acceptance Testing description

Contents

1 Introduction

1.1 Purpose and scope of document

The purpose of this document is to describe the planned testing processes for the Snowledge application (fi: Lumisovellus), providing a greater understanding to the different project stakeholders, such as the customer Pallaksen Pöllöt and the project team. Since the task is to rewrite the application according to the North Star vision, also the testing approaches and methodologies have been refined to correspond to this new ideology.

The Snowledge application will be tested using different test types. Unit tests are used to ensure the functionality of individual application components. Integration, system, and end-to-end (E2E) tests are used to verify that the different components as well as the whole application works as intended. Testing will be carried out by the developers, considering everyone's remaining working hours for the project. Additionally, acceptance testing will be done together with the customer and possible end-users.

1.2 Product and environment

The product under development in this project is the Snowledge web and mobile application, as already above mentioned. The main features of the Snowledge application are rescue functionality and representing snow conditions using the map and the segments in it. Due to project scope changes mentioned in the Project Plan document, the focus of this project work is mainly on recreating the web and mobile frontends according to the North Star vision. Another project group, later referred to as G26, is working on the backend implementation, in addition to developing an integration to the Suunto sports watches.

Since the development of the frontends and the backend have been separated into different project groups, it is increasingly important to test these components to a high standard. Testing them will be done individually, but also together after integration. The main responsibility for testing the frontends will be on this project work, being G05, while the focus of tests that G26 performs will be on the backend and their Suunto integration. It is a shared responsibility between the two groups that these components work together as planned.

1.3 Project constraints related to testing

The testing related project constraints are limited to the map data provider Mapbox and emergency calls. When the testing is performed, it should be ensured that the Mapbox API limits are not exceeded. Crossing the API limits would result in unwanted costs for the customer. Also, when the rescue functionality is tested, actual emergency calls should not be made.

1.4 Definitions, abbreviations and acronyms

Term	Definition
Android Internal App Sharing	Platform/tool for internally sharing application versions still under development.
API	Application Programming Interface
Apple TestFlight	Platform/tool for inviting users to test beta versions of applications before releasing them to the Apple App Store.
CI/CD	Continuous Integration and Continuous Delivery/Deployment
Domainhotelli	Finnish domain and web hotel service provider.
E2E	end-to-end
EPIC	Large body of work that can be broken down to smaller pieces.
Flutter	Mobile application development framework.
G05	Name/identifier of the group of this project work. Main responsibility is to rewrite mobile and web frontends.
G26	Name/identifier of the other project group. Main responsibility is to rewrite backend and implement Suunto integration.
GitHub	Hosting platform for Git version control system.
GitHub Actions	CI/CD platform that allows you to automate your build, test, and deployment pipeline.
Gitleaks	Security scanning tool that helps you find potential security vulnerabilities in e.g. Git repositories.
ID	identifier
Lumisovellus	Name of the application under development in Finnish.
Mapbox	Provider of custom maps and related data.
North Star	Name of application rewrite vision, main guide on path forward.
Pallaksen Pöllöt	Name of project customer. Fell guides / ski school in Pallastunturi.
Playwright	Automation library for creating E2E tests for web apps.
QA	Quality Assurance
Snowledge	Name of the application under development in English.
Suunto	Sports watch brand, part of G26's project work.
Tavu.io	Finnish cloud service provider
Vitest	Name of testing framework
WIP	Work In Progress

2 Quality Assurance Process

In this project the quality assurance process is divided into manual and automated methods. Various tools and different test types are utilized to ensure that the application's expected functionality is reached. The two following chapters describe these two main process types.

2.1 Manual Quality Assurance Process

The manual quality assurance process consists of multiple different aspects. The main parts are manual tests, code reviews, demo sessions, and deployment environments for the WIP versions of the web and mobile applications.

Due to resource constraints some of the testing will be done manually. Basically, this means that some of the system and E2E tests will be performed locally by the project group members. Some of the below mentioned deployment environments will be utilized for these manual tests. These tests consist of some pre-defined actions, which can be part of e.g. user stories created based on the EPICs. The objective of these manual tests is to test the functionality of the whole system.

The above-mentioned testing is mainly allocated for the QA Sprint. However, these tests start already when the integration of the backend and frontend begins. By removing the mock backend and replacing it with the actual backend, it can already be seen if the integration is successful or not. The main testing related to the integrated frontend and backend will happen after integration and deployment to the before mentioned deployment environments. If for some reason the deployment is unsuccessful, these tests will be carried out locally.

Code reviews are performed on pull requests in the GitHub version control platform. These code reviews are performed by the development team or the individual members of it. After completing the CI/CD pipeline, no pull requests will be merged before all the different steps defined for it have passed.

The current state of the application is demonstrated to other project stakeholders at different stages. First, individual group members present the development progress to the rest of the project group. Secondly, the whole project group or individual members of it demonstrate the progress to the customer. Based on the feedback gathered during the demo sessions, the application can be improved. These iterative demo sessions allow to focus on specific application areas. Thus, the quality of the application can be assured to correspond to the objectives defined for it.

Additionally, as already mentioned in the Project Plan document, different kinds of demo versions of the application are deployed to separate platforms for the customer and possible end-users to test. For the web version of the application, it is planned that the deployment is done manually to https://demo.lumisovellus.fi using the existing Tavu.io and Domainhotelli platforms. For mobile applications, Android Internal App Sharing as well as Apple TestFlight will be used to provide the demo versions. By utilizing these demo environments that quality of the applications can be assured before deploying to the actual upcoming production environment.

2.2 Automated Quality Assurance Process

Since the application development is done on GitHub the most straightforward and natural approach to automated quality assurance is to utilize GitHub Actions. Utilizing these workflows allows us to define various tasks for the whole process.

The currently planned reusable workflow consists of the following main tasks: validate, build, test, security, and smoke. The validation task is done to ensure the code quality and type consistency before other tasks. The build phase is done to secure that the application can be built successfully. The test task runs the implemented tests and enforces the test coverage threshold. The security task runs vulnerability and secret scanning, while the final task performs smoke and end-to-end tests after every previous task has passed successfully.

Utilizing this kind of reusable workflow allows us to define alternate configurations for it. Three alternate configurations have been defined for pull requests and one for push action. These alternate pull request configurations are for frontend, release, and master branches. The push configuration has been defined for frontend branch.

When a pull request is targeted towards frontend branch, only the above-mentioned validate task is performed. A push action to front end branch runs the validate, test, and security tasks. Pull requests targeted at release and master branches perform all the above-mentioned workflow tasks with minor differences. A pull request to master branch runs both fast and heavy lint with test coverage threshold set to 80, while a pull request towards release branch performs only the fast lint and the test coverage threshold has been set to 70.

3 Testing process

Coding conventions and quality guidelines have been defined and enforced with the above-mentioned GitHub Actions workflows. Coding style is carried out via two stage linting process, being fast and heavy lint. The fast lint process focuses only on some specific parts of the repository, while the heavy lint performs a deeper and slower scan of the codebase.

3.1 General approach

The general strategy for testing is carried out using multiple different types of tests. Unit tests are used to ensure that individual components of the application work as intended. Integration and system tests verify that these individual components as well as the whole application work as expected. Additionally, acceptance testing is performed to ensure that the implementation quality corresponds to the expectations that the customer and the end users have set for the whole application. The acceptance testing is performed with the help of the customer and possible end-users.

The focus in testing will be on system, integration, and E2E tests. Different kinds of analytic assessments, such as evaluations of usability, performance, and reliability, will be conducted as supplementary activities. These assessments are not the main focus of the testing process and will be performed primarily through qualitative methods and

user feedback rather than formal functional testing or quantitative metrics. Nevertheless, any feedback regarding these aspects will be considered and at least noted for future use.

3.2 Definition of done and goals

The definition of done can be defined for individual application components, integrated components, and the whole application in similar terms. These are defined as done when the implemented test cases have passed for them, and the customer has accepted the functionality that they define. After that no further work is required for those features or components. However, if these individual components are affected by some bug, they need to be retested to ensure proper functionality.

Three main objectives have been defined for testing in this project. The first one is that all implemented features should work as intended. For those features that aren't fully completed it's OK that they aren't entirely functional. The second objective is to identify weaknesses or defects in the system and make sure that they are addressed to improve the overall quality and reliability of the application. The final objective is to ensure that the user experience of the new application is at least equivalent to that of the current implementation and is considered adequate by the customer.

3.3 Testing roles

The different testing responsibilities have been divided into separate testing roles. Individual developers are responsible for implementing the test cases while the whole project group is responsible for getting the test cases to pass and the application corresponds to the expected behaviour. The customer is involved in acceptance testing with possible end-users. These other end-users can be for example other members of the Pallaksen Pöllöt. The main idea about the end-users is that they are not involved in the main development process and they can provide an outsider kind of view to the application. In other words, they do not know what is happening under the hood, and they treat the application as a black box.

The backend of the application is developed by the other project group, G26. They are mainly responsible for testing the back end. However, in the end the overall responsibility of the functionality of the application is on both project groups.

3.4 Test schedule

The testing schedule can be broken down into the following stages: CI/CD pipeline, integration testing, system and E2E testing, bug fixing and re-testing, and finally result reporting.

The CI/CD pipeline should be completed as soon as possible, after which it will take charge over the responsibilities defined for it. The integration testing starts immediately when the actual integration of frontend and backend begins. System and E2E testing will begin in the QA Sprint at latest. It is hard to estimate how much time will be required for bug fixing and retesting before the bugs have appeared, but a rough estimate for the duration can be said to be one week.

This Test Plan document describes which, how, and when tests will be performed. The other related document, being the Test Report, showcases the outcomes of those tests. The deadline of the Test Report document is 12th of December 2025. Therefore, all tests must be carried out, and the results must be reported in written form before that date.

If some bugs appear during testing, time is allocated for fixing them and retesting the affected features.

3.5 Test documentation

The tests performed will be documented using manual and automatic methods. The test frameworks used in the web and mobile applications will provide automatically generated reports. In the case of the web app, the framework generates a full test report which contains step-by-step results and detailed information in case of failures. In the case of the mobile application the tool provides coverage reports as well as reports from executed tests. The CI/CD pipeline collects test result artifacts from the workflow.

The results from manual testing and some of the automated tests are reported using an Excel table. These are all integration and system testing test cases. The Excel table contains the following columns:

- Test Level
- Platform
- ID
- Name
- Area
- Description
- Preconditions
- Test Steps
- Expected Outputs

Additionally, the actual report of the test cases contains also columns called “Actual Result” and “Status”.

4 Test cases

This chapter outlines the different testing practices planned for the Snowledge application to ensure its functionality, reliability, and quality. Each subsection describes a specific test type—such as unit, integration, system, and acceptance testing—and how it contributes to verifying that the application meets its technical and user requirements.

4.1 Unit testing

Unit testing focuses on verifying the functionality of individual components of both the web and mobile applications in isolation. These tests are primarily written by the developers as part of their regular implementation, following the project’s established coding and quality conventions. The purpose of unit testing is to ensure that each function, class, and component performs as expected before being integrated with other parts of the system. The main areas covered by unit testing include utility functions, state management logic, and UI components that do not depend on external services or APIs.

Unit tests are executed automatically through the CI/CD pipeline using GitHub Actions. On the web side, the tests are implemented using Vitest, while Flutter's built-in testing tools are used for the mobile application. These automated tests are run as part of every pull request and push action to maintain consistent code quality and prevent regressions. The CI/CD workflow also enforces code coverage thresholds and collects reports as build artifacts, which are reviewed to monitor testing progress. The outcome of the unit testing phase forms the foundation for later integration and system testing, ensuring that the basic building blocks of the Snowledge application are stable and reliable before further stages of quality assurance.

4.2 Integration testing

Integration testing is primarily focused on the areas where the frontend most depends on the backend, particularly the rescue and map-related features (segments and observations). Because these parts represent the most complex data interactions, they receive the greatest testing focus. Rescue functionality has the highest integration bias, as it involves multiple connected users, real-time notifications, and backend-controlled state changes. The map, segment, and observation features form the second major focus area, since they combine large data loads, offline caching, and synchronization logic. Other modules, such as weather and Snower, receive lighter coverage because their interaction is comparatively straightforward and use external APIs.

4.3 System testing

System testing emphasizes realistic, end-to-end workflows, but there is a natural bias toward the map and rescue features, as these define the main use cases of the application. The map and segments are tested thoroughly to ensure that visual rendering, filtering, and observation creation all function reliably in both online and offline conditions. The rescue workflow receives extensive system testing due to its critical nature, involving location accuracy, notifications, and multi-user coordination. Supporting features such as Snower and weather are included in system testing, but with less emphasis, as they have lower functional risk and simpler dependencies.

4.4 Acceptance testing

Acceptance testing is carried out by the customer, Pallaksen Pöllöt, to ensure that the implemented features meet their expectations and agreed requirements. The customer is provided access to the latest demo versions of the application for independent testing. The web version is available at https://demo.lumisovellus.fi, while mobile versions are distributed through Android Internal App Sharing and Apple TestFlight. This allows the customer to test the application on their own time and provide feedback, which will be reviewed and incorporated into the final refinements before release.

Before the acceptance testing can be done the frontend and backend should be integrated together. Additionally, the different demo versions of the applications should be deployed on the platforms meant for them. The acceptance testing is scheduled to happen during Sprint 5.

The customer should reserve time for testing and do it as soon as possible when the demo versions are available. The customer will perform the acceptance testing independently. Any bugs found during the testing process are written down. These bugs

and other possibly required changes will be discussed together with the project group in a meeting scheduled after the acceptance testing has been done.

The found bugs and required fixes are prioritized and done depending on their severity. The fixes will happen during the QA Sprint, i.e. the 6th Sprint at latest. However, the fixes can be omitted if:

- That has been agreed with the customer, e.g. something else with a higher priority is ongoing at the same time.
- There isn't enough time to do them. Nevertheless, the found bugs and required fixes are documented for future use.

5 Adopted Tools

Various tools are utilized during testing. On the web side, Playwright and Vitest are used, while the Flutter built-in tools are utilized on the mobile side. The CI/CD pipeline has been created using GitHub Actions. It executes automated tests, which consist of unit tests as well as some integration, system, and E2E tests. As mentioned before, not all tests have been automated. One tool within GitHub Actions that is worth mentioning is Gitleaks. It is used to detect and prevent security vulnerabilities in the code base.

6 Open issues

None at the time of writing.

APPENDIX A: Table of Test Cases